Quiz 9: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 9 is Thursday Oct 29, at the end of class. About 15 minutes, 25 points.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed. A formula sheet is handed out with the quiz, so nothing needs memorizing.

> ❗ There is no coding section
>
> Quizzes 3 through 7 ended with a "find the bugs in this script" exercise. That is gone for good. Part D is **short answer**: one or two sentences explaining why something behaves the way it does.
> Reading code is a skill this course uses constantly. Debugging syntax is not one it grades.

Scope

The Oct 22 lecture. The recording is 1 hour 44 minutes. It reviews tokenization and embeddings, finishes the sequence deck through gated recurrence, and gets through attention as far as multi-head. It stops there, before the positional-encoding formulas, the block, LayerNorm, and the encoder/decoder split. Student presentations start at about 01:43.

On the quiz:

Sequence data, as far as it was taught

- padding: sequences in one batch share a length T, and T can change from batch to batch. Pad tokens are meaningless.
- sorting a batch by length, so short sequences are not padded out to a long one
- masking means "ignore this token." The two reasons named in class: the token is padding, or it is the answer the model is not allowed to see yet
- a 1D convolution is a kernel that slides along the sequence. A kernel of $\frac{1}{2}, \frac{1}{2}$ is a moving average. The kernel is learned
- the recurrence: a hidden state updated from the current input and the previous state, one token at a time
- generation produces one token at a time
- backpropagation through time: the gradient is a product along the steps, so it decays exponentially, and a tiny gradient means that step is not learning
- what gating fixed. Partly the reach of the gradient, when the forget gate is near 1. Not the one-step-at-a-time limit
- self-attention costs n^2 because every position is compared with every other. A recurrence costs one step per token

Attention

- **query, key, and value.** The query asks, the key answers, the value is what gets mixed in. The dog's query with furry's key is a different number from furry's query with the dog's key
- the scaled dot product, $QK^\top/\sqrt{d_k}$, then softmax, then times V

- why the square root is there: it keeps the dot products from saturating the softmax, so the gradient can still flow
- a **padding mask** is $-\infty$ on pad positions, added before the softmax, because $e^{-\infty} = 0$
- **multi-head attention.** One head is not enough. h heads split a width d into slices of d/h. That split has to come out even. More heads do not add parameters
- **why position has to be added.** A dot product does not know where a token sits, so "dog" third and "dog" hundredth give the same product. A positional embedding is a second lookup, one row per position, added to the token embedding. The sinusoidal formula was not reached

From Lab 8

- the target at position t is the token at $t + 1$
- a causal mask puts $-\infty$ on the future, the first row lands on itself, and every row still sums to 1
- generation appends a sampled word and runs again, one forward pass per new word. Teacher forcing scores every position in one pass, because the true words are already there

Not on the quiz. The sinusoidal formula, learned-versus-sinusoidal extrapolation, the feed-forward network, LayerNorm versus BatchNorm, and which of encoder, decoder, or both a model uses. Also off: the receptive-field formula for a stack of 1D convolutions, writing `collate_fn`, GRU internals, and reproducing every LSTM gate from memory. He walked the LSTM diagram and said the exact arrangement is not the thing to memorize. Next week's large language model material is off too.

Part 1: Padding

A batch has to be a rectangle, because a tensor has a shape. Sentences are not the same length. **Padding appends a reserved token until every sequence reaches the length of the longest one in the batch.**

$$T = \max_i \ell_i, \qquad \text{cells} = N \times T, \qquad \text{padding share} = 1 - \frac{\sum_i \ell_i}{N \times T}$$

Worked example. Lengths 3, 7, 12, 2: $T = 12$, cells $= 48$, real tokens $= 24$, so **50% of the batch means nothing**. In Lab 7 a random batch of 8 real sentences was 52.1% padding.

T **comes from this batch, not from the dataset**, and changes every batch. Sorting by length first puts the long sequences in one batch and the short ones in another, so the short batch is not padded out to the long one's T. The lecture pointed at `collate_fn` and did not walk the code.

By convention `<pad>` is id 0 and `<unk>` is id 1, with real tokens from 2. If you build the vocabulary without reserving those, your most frequent word gets id 0 and becomes indistinguishable from padding.

Part 2: Masking

The lecture's version of this is short. A mask says which tokens to ignore. The two reasons he named: the token is padding, or it is a future token the model is not allowed to see. The arithmetic below is Lab 7. It is here so the padding story stays in one place. Quiz 9 will not ask you to recall the percentages.

A mask is a boolean tensor the same shape as the batch, marking which positions are real. It is `ids != PAD`. Without one, every layer downstream reads `PAD` as a word.

The exact error, which is the thing to remember

Because `padding_idx` pins the pad vector to zero, the unmasked mean is the sum of the real vectors divided by T, while the correct mean divides by ℓ:

$$\text{naive} = \frac{v}{T}, \qquad \text{masked} = \frac{v}{\ell} \qquad \Rightarrow \qquad \text{relative error} = 1 - \frac{\ell}{T}$$

Lab 7 measured this and the measured column matched the formula to the digit for all eight rows, with a correlation between length and error of exactly -1.

length	T	error
2	24	91.7%
12	24	50.0%
24	24	0.0%

Why a masking bug hides

It damages short examples in proportion to how short they are, and leaves the longest member of each batch untouched. So the validation average looks only mildly wrong. Wild behaviour on short inputs and clean behaviour on long ones points at masking before it points at the model.

The three places

where	what it looks like
pooling	divide by the real count, not T
the loss	`ignore_index=0`
attention	pad positions go to $-\infty$ before the softmax

Forgetting any one is silent. Nothing raises.

On the loss, Lab 7 compared two hypothetical models on a batch that was 52.1% padding. The one that got **every real word wrong** and every pad right scored an unmasked loss of **4.306**; the one that got every real word right scored **4.681**. Under an unmasked loss the useless model wins, which follows from more than half the target cells being `PAD`.

Part 3: Building the batch, and bucketing

A `Dataset` returns one example. Default collation stacks them and needs matching shapes, which ragged sequences do not have, so it raises.

`collate_fn` receives the list and returns whatever the model wants: pad to the longest member of *this* batch and hand back the mask alongside the ids.

Bucketing means sorting by length first, so batches become internally uniform and most padding disappears. Lab 7, on 2,048 sentences:

batch size	shuffled	sorted
8	60.2% padding	2.2%

batch size	shuffled	sorted
32	73.5% padding	10.1%

In practice you shuffle, take a pool of a few hundred, sort inside the pool and batch from that, because sorting the whole dataset means every epoch sees the same batches.

The one place order is not yours to change: a time series. Split it on time, train on the past, validate on the future. A random split lets the model see tomorrow while it learns yesterday.

Part 4: 1D convolution

`nn.Conv1d` takes (N, C, T), note the order, and **accepts any T with the same weights**. So variable length was never the real objection to using convolution on text. Locality is.

A k-wide kernel widens the receptive field by $k-1$ per layer, so L stacked layers see $L(k-1)+1$ positions. With $k = 3$ that is $2L + 1$.

> **Likely question.** How many layers of kernel 3 before one position can see a token 100 away? You need 101 positions in view, so $2L + 1 \geq 101$ and $L \geq 50$. Attention needs one layer.

1D convolution is **current practice, not history**: a convolutional front end before a speech transformer, Conformer blocks, dilated causal convolutions for time series. Use it where the pattern is local.

Part 5: The recurrence, and why it breaks

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)\,, \qquad h_0 = \mathbf{0}$$

The same weights run at every step. That is **weight sharing**, the same idea as a kernel sliding over an image except over time, and it is what lets one model take any length.

It is also the problem: **the only route from the first token to the last is along the W_h arrows, one multiplication per step, with no other path.**

Reading the output

`nn.RNN` returns `(output, h_n)`. `nn.LSTM` returns `(output, (h_n, c_n))`. With `batch_first=True`, `output` is (N, T, H): the hidden state h_t at every step. The cell state c_t is not in `output`. Which part you read makes it a different task. **With padding, `out[:, -1]` is the state after reading pad tokens**, not after reading the sentence.

Generation and teacher forcing

Predict one token, append it, run again: one forward pass per token. Training does not work that way, because feeding the model its own early predictions compounds errors, so you feed the true previous token instead. That is **teacher forcing**, and it is why training is parallel over positions while generation is not.

Failure one: the gradient does not reach back

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

A product of $T-t$ terms. Smaller than 1 and it goes to zero. **No choice of weights makes a product of many terms behave like a sum.** Measured: the vanilla RNN keeps **0.53** of the gradient per step, a usable reach of about 20 steps.

Exploding is the easy half, you clip the norm. Vanishing is the hard half, because there is nothing to clip.

Say the cost precisely: it is not that the model is wrong about long sequences, **it is that it cannot learn anything that depends on one**, because the signal that would teach it never arrives.

Failure two: one step at a time

h_t needs h_{t-1}, so the steps cannot run together, no matter the hardware. Measured: an LSTM at **16.00 ms** against attention's **7.63 ms** on 1,024 tokens, and attention does *more* total arithmetic.

What the gates compute

c_t is the cell state, the memory. h_t is the hidden state, what the cell exposes. x_t is the input at this step.

Every circle reads h_{t-1} and x_t. It concatenates them, multiplies by its own weight matrix, adds its own bias, and squashes:

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \\ i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \\ \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c) \end{aligned}$$

f_t is the forget gate, i_t the input gate, and o_t the output gate. All three lie in $(0, 1)$. $\tilde{c}_t$ is the candidate. It uses tanh, so it lies in $(-1, 1)$.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \qquad h_t = o_t \odot \tanh(c_t)$$

With f_t near 1, $c_t \approx c_{t-1}$, so the gradient along that track is multiplied by roughly 1 instead of a weight matrix. That is the residual connection's trick, twenty years earlier.

	fixed?
gradient reach	partly: 28 steps against 20, still exponential
one step per token	not at all

Part 6: Query, key and value

This is the one new idea in the transformer, and the most likely thing to be asked about. Every token produces three vectors from its own embedding, each by its own learned linear layer:

- **query** q: what this token is looking for
- **key** k: what this token offers to anyone looking
- **value** v: what this token contributes if attended to

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_{\mathbf{Q}}, \qquad \mathbf{K} = \mathbf{X}\mathbf{W}_{\mathbf{K}}, \qquad \mathbf{V} = \mathbf{X}\mathbf{W}_{\mathbf{V}}$$

A token's attention to another is its query dotted with that token's key. Those scores become weights, and the output is the weighted sum of the values.

The library analogy: the query is your request, the keys are the spine labels, the values are the books.

In **self-attention** all three come from the same sequence. In **cross-attention** the queries come from the decoder and the keys and values come from the encoder.

Part 7: Scaled dot product attention

$$\mathbf{A}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = S\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

Four steps: score every pair, divide by $\sqrt{d_k}$, softmax each row, multiply by the values.

Why the square root

A dot product of two d_k-dimensional vectors is a sum of d_k terms, so its spread grows like $\sqrt{d_k}$. Measured at $d_k = 512$: the unscaled softmax puts **91%** of its weight on one token, against **3%** when scaled.

Saturation is not the real cost. A saturated softmax has almost no gradient, so the layer stops learning: the scaled version delivers **6 times** more gradient to its logits.

The attention matrix

Sequence by sequence. Entry (i, j) is how much token i attends to token j, and **every row sums to 1**. It is $n \times n$, which is where the quadratic cost comes from, and it is recomputed from the content at every layer.

The output of an attention layer is the same shape as its input, (N, T, d).

Part 8: Multi-head attention

One attention layer produces **one** weight matrix, so it expresses one notion of relatedness per layer. Language has several at once.

Multi-head attention runs h attention operations in parallel on different learned projections of the same input, then concatenates.

Heads are free. A width of 512 split across 4 heads gives each head 128 dimensions. PyTorch will refuse the layer if d is not divisible by h. The projection matrices cost the same number of parameters at 1 head and at 8, because the slices got thinner rather than the vector getting wider. What you buy is several different blends of the same values.

Part 9: Position

A dot product does not know where a token sits. The query for "dog" dotted with the key for "furry" is the same number whether "dog" is the third token or the hundredth. Shuffling the sentence does not change any of those products, so the model cannot tell "the dog bit the man" from "the man bit the dog."

Positional encoding is a second embedding, one row per position, added to the token embedding. That is as far as the lecture got. The sinusoidal formula, and the difference between a learned table and a fixed one, were not reached.

Part 10: Masks inside attention

A padding mask stops pad tokens from affecting the weights. It is a matrix of zeros, with $-\infty$ where the token is padding, added to $QK^\top$ before the softmax. $e^{-\infty} = 0$, so that position drops out.

The other mask is the one in Lab 8. **A causal mask stops a position from seeing later tokens**, which is the "do not give it the answer" point from the start of the lecture. Same trick, $-\infty$ before the softmax. The first token has nothing before it, so its row lands entirely on itself, and every row still sums to 1 because the softmax runs after the mask.

Not reached on Oct 22

The lecture stopped during multi-head attention. These are in the deck and are not on Quiz 9:

- the sinusoidal positional formula, and what a learned position table cannot extrapolate past
- the feed-forward sublayer, and the block wrapped in a residual and LayerNorm
- why LayerNorm is used instead of BatchNorm
- encoder only, decoder only, and encoder-decoder, and which models are which

---

How to prepare

1. **Define a query, a key, and a value in one sentence each**, and say what number decides how much one token attends to another. If you learn one thing, learn this.
2. **Say why the square root is there.** The reason he gave is that the gradient still has to flow. Saturation of the softmax is the mechanism.
3. **Say what a padding mask does to a softmax**, including why the fill value is $-\infty$.
4. **Say what gating fixed and what it left alone.** Partly how far the gradient reaches. Not the one-token-at-a-time limit.
5. **Say why self-attention is** n^2 and a recurrence is one step per token.
6. **From Lab 8**, say what the target at each position is, why a causal mask's rows still sum to 1, and why generation takes one forward pass per new word while teacher forcing takes one pass for the sequence.

The formula sheet has the attention equation, the recurrence, the product for backpropagation through time, the LSTM update, and the padding formulas. It does not say what any of them are for. That part is the quiz.